

جـامعة بيروت العربية
BEIRUT ARAB UNIVERSITY

Faculty of Science

Department of Mathematics and Computer Science

Operating Systems (CMPS442)

FCFS Scheduling Algorithm & Banker's Algorithm

Student Name: **Student ID:**

Nour Ajami 202302485

Supervisor:

Dr. Mohamed Al Saleh Supervisor

T.A. Sandi Zein El Zein Lab Instructor

May 17, 2026

Introduction

Simulator Description

This project utilizes the OS Algorithm Simulator on Android. The simulator can manually add processes and test out various process scheduling algorithms as well as try out different methods to avoid a deadlock. For each process that is added to the simulator, there are fields to input the process's burst time, arrival time, and its priority. The step-by-step simulation shows out the various steps the processes go through in the CPU as well as the waiting time and the turnaround time for each process. Additionally, it displays the amount of CPU time allocated for each process as well as the current state of each process.

For the scheduling part of the project, I implemented the First Come First Served (FCFS) scheduling algorithm using the simulator. Each process was created with the burst time, arrival time and the priority for the process.

For the part of deadlock avoidance, the Banker's Algorithm was implemented using the same simulator. The inputs for this part were the Allocation matrix, the Maximum matrix, and the Available resources vector. Then the Need matrix was calculated, and it was checked whether the system was in a safe state or not.

The simulator also allowed me to transform what I have studied in class to a working simulator that shows the processes being worked in detail.

Project Objectives

The project contains two parts.

In the first part, 12 processes are used to test a simulation of the processes' execution under the FCFS scheduling algorithm. The processes in the simulation have different arrival times and burst times, and the simulation records waiting time and turnaround time for each process and graphs them and cross plots them against each other for all processes.

The second part for this project is to work with the Banker's Algorithm for managing processes using a fixed set of resources. This will involve

creating a system with 4 processes and 3 different types of resources and then calculating the Need matrix for various possible allocations of resources to the processes and then determining whether the system is in a safe state for the allocations tested.

Problem Statement and Experimentation

Part A: FCFS Scheduling Algorithm

Problem Statement

First Come First Served (FCFS) is a non-preemptive CPU scheduling algorithm. Processes are executed strictly according to their arrival time. The process that arrives first gets the CPU first and continues execution until completion before the next process starts.
The experiment uses 12 processes with different arrival and burst times. The goal is to observe CPU execution order, waiting times, and turnaround times using FCFS scheduling.

Process Configuration



Figure 1: Initial FCFS process configuration inside the simulator.



Figure 2: Complete list of the 12 configured processes used in the FCFS simulation.

PID	Arrival Time	Burst Time	Priority
P1	0	8	2
P2	2	6	1
P3	4	10	3
P4	6	5	4
P5	8	7	2
P6	10	9	5
P7	12	5	1
P8	14	6	6
P9	16	8	3
P10	18	5	7
P11	20	7	4
P12	22	5	2

Simulation Steps

At time 0, there only is one process available in the ready queue, process P1. Because FCFS is a queue driven process scheduler (as opposed to a thread driven process scheduler), the FCFS process scheduler will give the CPU to process P1 and then process P1 will continue to run until it completes.

Processes continue to arrive and are placed at the end of the ready queue. Each process waits for the earlier processes to finish their CPU execution.

The following steps are shown in a simulation of the execution of the processes in FCFS order.



Figure 3: Execution of process P1 from time 0 until time 8.



Figure 4: Execution of process P2 after completion of P1.



Figure 5: Execution of process P3 inside the FCFS queue.

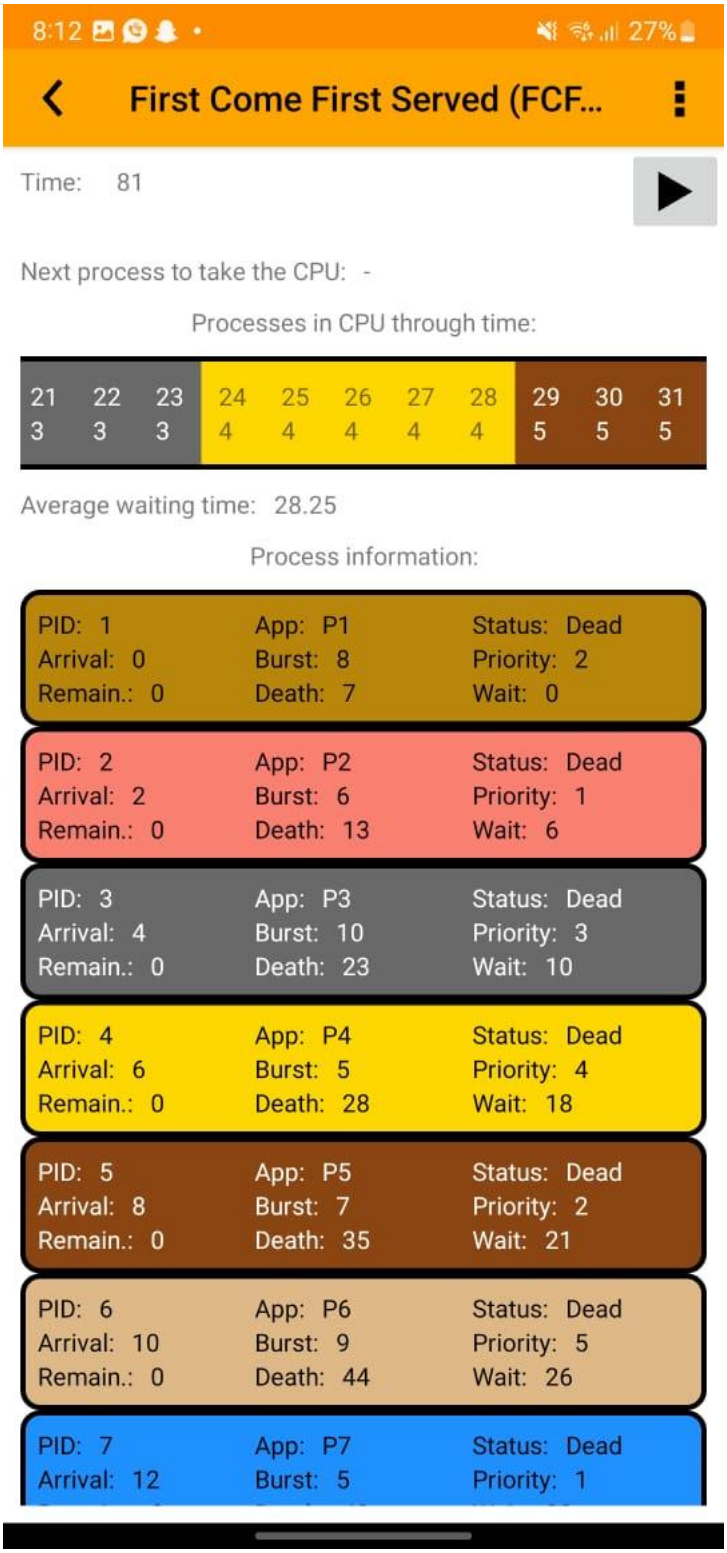


Figure 6: Execution of process P4 after waiting in the ready queue.

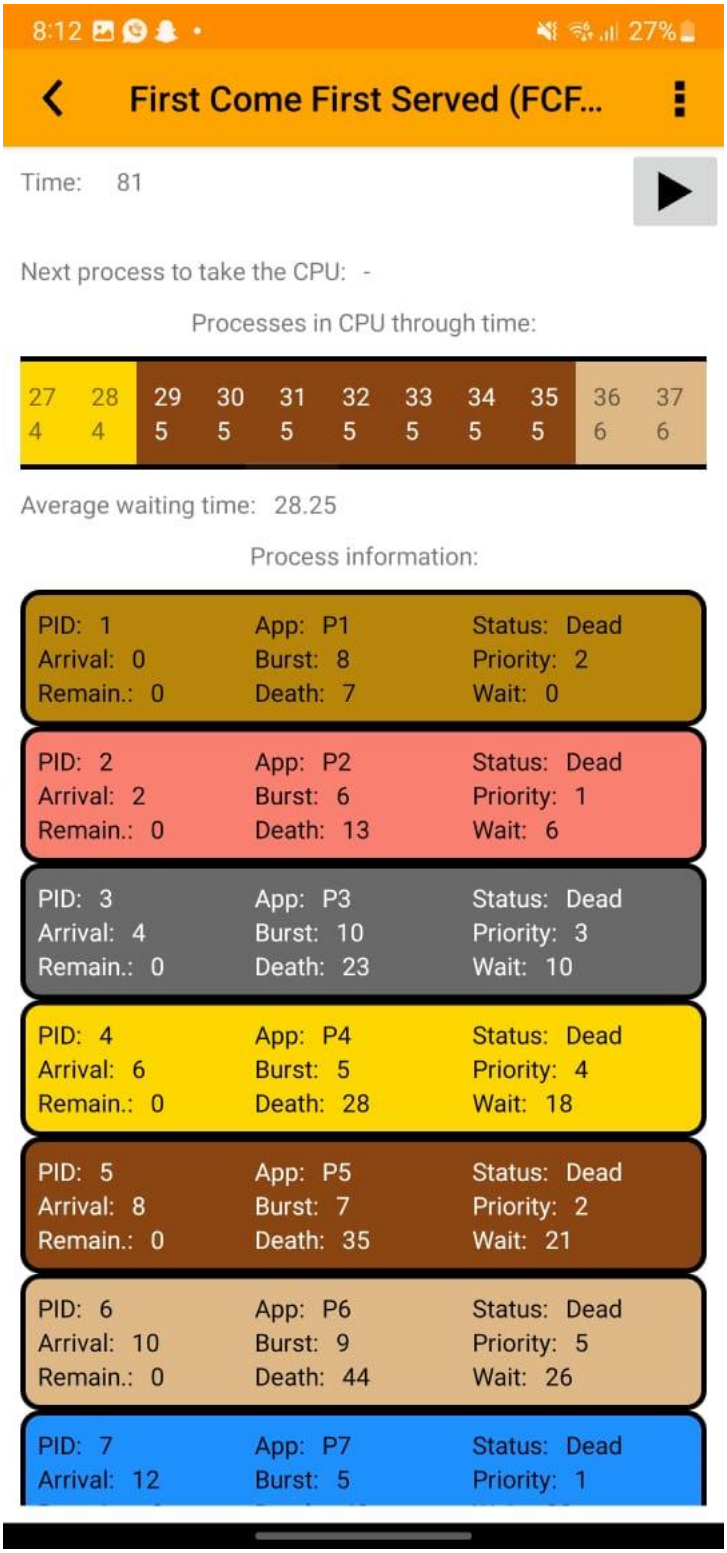


Figure 7: CPU execution sequentially according to process arrival order.



Figure 8: Middle stage of the FCFS simulation showing completed and waiting processes.



Figure 9: Execution of later arriving processes inside the FCFS queue.



Figure 10: FCFS scheduling timeline during process switching.



Figure 11: Execution of processes P9.



Figure 12: Execution of process P10.



Figure 13: Execution of process P11.



Figure 14: Execution of final process P12.



Figure 15: Final state of the FCFS simulation after all processes completed using the FCFS scheduling algorithm.

Results

FCFS Formulas:

Turnaround Time (TAT) = Finish Time – Arrival Time
Waiting Time (WT) = Turnaround Time – Burst Time

PID	Arrival	Burst	Finish	TAT	WT
P1	0	8	8	8	0
P2	2	6	14	12	6
P3	4	10	24	20	10
P4	6	5	29	23	18
P5	8	7	36	28	21
P6	10	9	45	35	26
P7	12	5	50	38	33
P8	14	6	56	42	36
P9	16	8	64	48	40
P10	18	5	69	51	46
P11	20	7	76	49	49
P12	22	5	81	59	54

Average Waiting Time = 28.25

Banker's Algorithm

Problem Statement

Banker's Algorithm is a method for deadlock avoidance. The Banker's Algorithm tries to determine whether a given set of pending process requests for resources will result in a safe state. That is, the algorithm tries to determine whether there is a safe assignment of resources to the processes requesting them for completion of their processing. The Banker's Algorithm is used before a process is assigned any resources. The algorithm determines whether all processes can proceed safely.

System Configuration

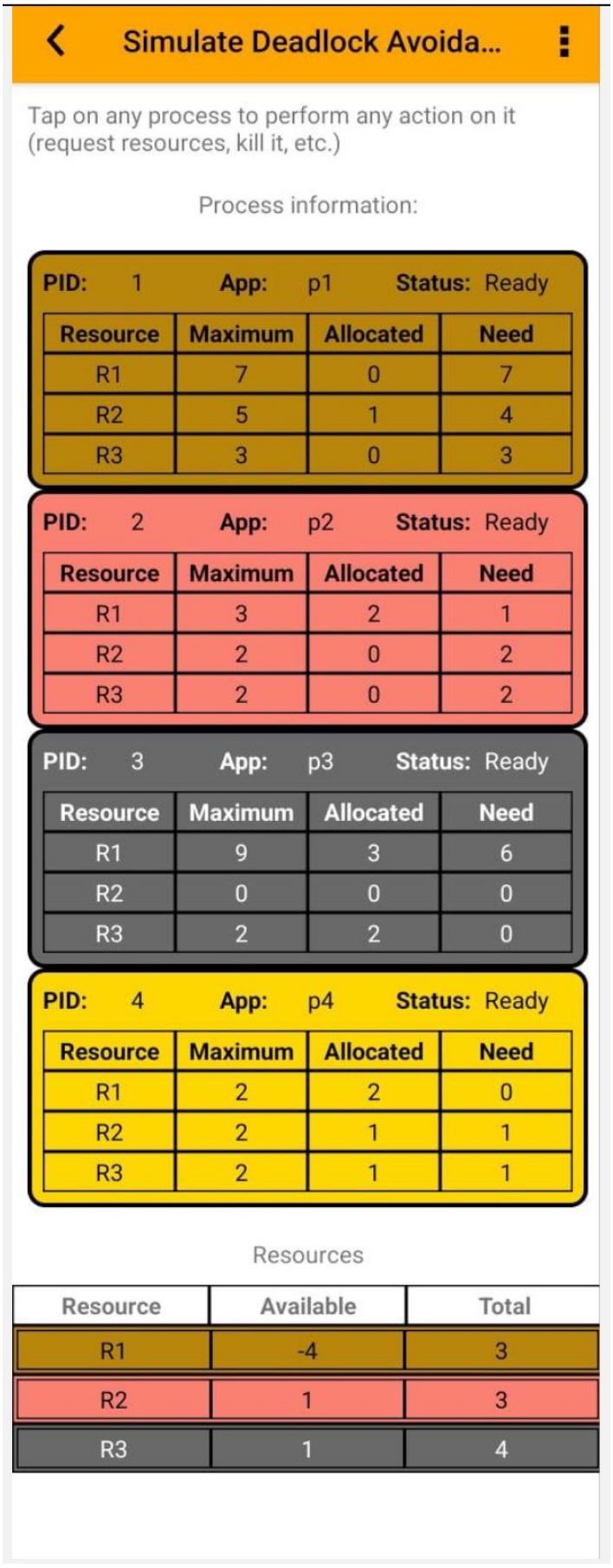

Simulate Deadlock Avoida...
Tap on any process to perform any action on it (request resources, kill it, etc.)
Process information:
PID: 1 App: p1 Status: Ready
Resource Maximum Allocated Need
R1 7 0 7
R2 5 1 4
R3 3 0 3
PID: 2 App: p2 Status: Ready
Resource Maximum Allocated Need
R1 3 2 1
R2 2 0 2
R3 2 0 2
PID: 3 App: p3 Status: Ready
Resource Maximum Allocated Need
R1 9 3 6
R2 0 0 0
R3 2 2 0
PID: 4 App: p4 Status: Ready
Resource Maximum Allocated Need
R1 2 2 0
R2 2 1 1
R3 2 1 1
Resources
Resource Available Total
R1 -4 3
R2 1 3
R3 1 4

Figure 16: Banker's Algorithm simulation showing Maximum, Allocation, and Need metrics.

Maximum Matrix

Process	R1	R2	R3
P1	7	5	3
P2	3	2	2
P3	9	0	2
P4	2	2	2

Allocation Matrix

Process	R1	R2	R3
P1	0	1	0
P2	2	0	0
P3	3	0	2
P4	2	1	1

Available Vector

- R1 = 3
- R2 = 3
- R3 = 4

Need Matrix

Formula: Need = Maximum – Allocation

Process	R1	R2	R3
P1	7	4	3
P2	1	2	2
P3	6	0	0
P4	0	1	1

Safety Algorithm and Safe Sequence

The safety algorithm starts with the available vector:

Work = [3,3,4]

The algorithm now looks for a process with a Need less than or equal to the current available resources. This process can now complete as it was using resources that have since been returned to the available vector after that process's completion.

Step	Process	Need	Work Before	Allocation	Work After
1	P2	[1,2,2]	[3,3,4]	[2,0,0]	[5,3,4]
2	P4	[0,1,1]	[5,3,4]	[2,1,1]	[7,4,5]
3	P1	[7,4,3]	[7,4,5]	[0,1,0]	[7,5,5]
4	P3	[6,0,0]	[7,5,5]	[3,0,2]	[10,5,7]

Safe Sequence:
P2 → P4 → P1 → P3
The system is in a safe state.

Discussion

Part A – FCFS

The is a non-preemptive process, each process runs until it is completed before the next process in FCFS simulator creates a simulation where processes are executed in order of their arrival time. Because FCFS the ready queue can begin to run.

The waiting time for the processes increased as their arrival time increased. This is because the processes that arrived first had the lowest waiting time. P1 had a waiting time of 0 because it was the first process in the ready queue. P12 had the highest waiting time for this reason as well. P12 arrived late and therefore had to wait for all the processes that arrived before it to complete their execution.

The convoy effect can also be observed by the long processes P3 and P6 which hold up the smaller processes that are running behind them for a long time and thus increase their waiting time.

When we look at the theoretical view of the FCFS scheduling algorithm in class and our simulator, it behaves as we have stated before in class. The FCFS scheduling algorithm is easy to implement, but it is not the best choice in terms of waiting time and also turnaround time.

Part B – Banker's Algorithm

The Banker's Algorithm simulation demonstrated deadlock avoidance step by step. First, the Need matrix was computed from the Maximum matrix and the Allocation matrix.

From the above simulation we have concluded that the system is in safe state, and the safe sequence of processes will be

P2 → P4 → P1 → P3.

When a process finished its execution, all the resources which were simulation of the Banker's Algorithm was very satisfactory. It was very good to see how resources can allocated to this process were put back in the available vector, so that other processes could continue their execution

Conclusion

This project explores two core aspects of an operating system: CPU scheduling and deadlock avoidance.

In Part A of the project the simulator for the FCFS scheduling algorithm was successfully tested on 12 processes with different arrival times and burst times. The simulation showed the expected behavior of the scheduling algorithm. The processes are executed in the order of their arrival. Processes which arrive later have to wait for earlier processes to complete because the CPU can only be used by one process at a time. This results in longer waiting times for the later arriving processes.

In Part B, I made use of the Banker's Algorithm in order to test if the system was in a safe state for a given allocation of resources. The Need matrix was calculated successfully and then the system was proven to be safe in the following order: P2 → P4 → P1 → P3.

P2 → P4 → P1 → P3

I connected theoretical knowledge of OS with practical simulations for better understanding of CPU scheduling and for deadlock avoidance using algorithms.